

Lecture 9 - Sampling Review Rejection, Ancestral, MCMC

DS 4420: Machine Learning and Data Mining 2

Eric Gerber

`e.gerber@northeastern.edu`

Northeastern University, Boston, USA



Today

- 1 Sampling Review
- 2 Rejection
- 3 Ancestral
- 4 MCMC
- 5 MCMC: MH
- 6 MCMC: Gibbs

Today

- 1 Sampling Review
- 2 Rejection
- 3 Ancestral
- 4 MCMC
- 5 MCMC: MH
- 6 MCMC: Gibbs

Updates

- Check-in at a Project Help Session **this week**
- Quiz 7 (on Lecture 9 material) planned for **Wednesday, Apr. 1**
- Homework 6 due **Monday, Apr. 6 by 11:59 pm**

Topics for Today:

- 1 Sampling Review
 - Overview
 - Rejection Sampling
- 2 Ancestral Sampling
- 3 MCMC
 - Metropolis Hastings
 - Gibbs Sampling

Push-Up Counter:

- Section 1: 17
- Section 4: 12

Motivation

Recall that our goal in Bayesian machine learning is to understand the posterior distribution of an unknown parameter, θ **given** the data we collect, y :

$$p(\theta|y) = \frac{p(y|\theta)p(\theta)}{p(y)}$$

- $p(y|\theta)$ is the **likelihood**, we **choose** this based on how we believe our data behave
- $p(\theta)$ is the **prior**, we **choose** this based on how we believe θ behaves
 - Together, they make up: $p(y, \theta) = p(y|\theta)p(\theta)$, the **joint** distribution
- $p(y)$ is the **evidence/marginal** and we do **NOT CHOOSE** this:

$$p(y) = \int_{\theta} p(y, \theta) d\theta$$

Motivation

We have mostly dealt with situations where $p(y)$ was reasonably (if not easily) found. If it is **not** easy, we have two main tricks for figuring out the posterior $p(\theta|y)$ without needing to **explicitly** determine $p(y)$:

- Use a **conjugate** prior
 - If we do this, we know that the form of the posterior will be the same as the prior; we simply need to derive (or intuit) what the updated parameters will be
 - **We've been doing this**
- Use **sampling** to estimate $p(\theta|y)$
 - We are doing this today

Using Samples to Approximate Integrals

We already should have some notion of how samples of data can approximate integrals. Note:

- The expected value of a probability distribution y is:

$$E(y) = \int y p(y) dy$$

- The approximation of the expected value is the sample mean of n samples from y :

$$\bar{y} = \sum_i^n y_i \left(\frac{1}{n} \right)$$

Above, you can see that we approximate the integral by summing (instead of integrating) and weighting each y value by $\frac{1}{n}$ (instead of $p(y)$).

Sampling

Sampling is the generation of (simulated/synthetic) data from a distribution, $p(y)$. There are some built-in NumPy functions which do this for common distributions. For example, see **sampling_mcmc.ipynb** and **sampling_mcmc.r** where we use the **np.random.normal()** and **rnorm()** functions to sample from a normal distribution.

- 1 Look at the first 10 numbers of 1000; at first they look completely random without pattern
- 2 However, if you plot all 1000 in a histogram, you see the clear pattern of a Gaussian distribution
- 3 We didn't have data, but **from a known $p(y)$, we have simulated some!**

Python and R's Common Distributions

NumPy has random number generators under many different common $p(y)$'s, which we have already seen. (These are listed in **sampling_mcmc.ipynb**) R has many distributions that are shipped with the base version.

So What?

The benefit of sampling comes from when the target function, $p(\cdot)$, is too difficult to integrate directly. More precisely, if we want to find:

$$\int_{\theta} p(y, \theta) d\theta$$

Couldn't we:

- Generate **joint** samples from $p(y, \theta)$ using a sampling technique.
 - Note that because $p(y, \theta)$ is a joint distribution, we will be generating n samples of (y_i, θ_i)
- Use those samples to approximate the posterior, $p(\theta|y)$?

Motivating with the Bernoulli-Beta

Recall our initial Bayesian model example.

$$y|\theta \sim \text{bern}(\theta)$$

$$\theta \sim \text{beta}(\alpha, \beta)$$

Where y was whether or not your crush texted you back ($y \in \{0, 1\}$) and θ was the probability they are interested in you. After some work, we showed that the posterior was:

$$p(\theta|y) = \frac{p(y, \theta)}{p(y)} = \frac{p(y|\theta)p(\theta)}{\int_{\theta} p(y, \theta)d\theta} =$$
$$\frac{\theta^y(1-\theta)^{1-y} \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1}(1-\theta)^{\beta-1}}{\int_{\theta} \theta^y(1-\theta)^{1-y} \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1}(1-\theta)^{\beta-1}d\theta} = \frac{\theta^{y+\alpha-1}(1-\theta)^{1-y+\beta-1}}{B(y+\alpha, 1-y+\beta)}$$

- **Important:** once you collect the data y , the evidence, since it has nothing to do with the unknown θ , **IS A CONSTANT**, in this case either $c_1 = p(y=0)$ or $c_2 = p(y=1)$
- If we can simulate data under $p(y, \theta)$, then we can estimate $p(y)$ simply by counting up the number of zeros and ones in our synthetic data!

Before We Go Too Fast

We'll revisit this in a second, as sampling from joint distributions (which we need to do here) can be tricky. First, let's simply remind you of that there are several commonly used ways to sample from univariate distributions:

- Inversion Sampling
- KDE Sampling
- Importance Sampling
- **Rejection Sampling**

Some of you did these in DS 4400, but they are not all necessary for our purposes. We'll cover **rejection sampling** again to make sure everyone has a basis for continuing.

Today

1 Sampling Review

2 Rejection

3 Ancestral

4 MCMC

5 MCMC: MH

6 MCMC: Gibbs

Rejection Sampling

- **GOAL:** to sample from a complex $p(y)$
- **ASSUME:** we can sample from a simpler distribution $q(y)$ that, when multiplied by a constant k , covers the $p(y)$
- **KEY OPERATION:** accepting or rejecting a sample based on the ratio of $p(y)$ to some $k \times q(y)$, where $k \times q(y) > p(y)$

Basic Ideas

Here is a trivial (almost nonsensical) case: suppose you want to sample from a continuous uniform distribution with lower bound $a = 0$ and upper bound $b = 2$ so that, $p(y) = \frac{1}{2}$. (Draw a picture).

- Pick a distribution that is easy to sample from (any of the common ones, perhaps the uniform?) and call it $q(y)$
 - So, in our trivial example, $q(y)$ is **also** uniform 0 to 2
- Multiply this by some constant k such that it is always larger than $p(y)$
 - So, for our example, perhaps $k = 2$ so that $kq(y) = 1$

Why?

Pick any point in the range of y . If you propose generating a sample from that point with probability $\frac{p(y)}{kq(y)}$, what do you get?

Basic Ideas

Extend this to a slightly more complicated example. Imagine you have

$p(y) = \begin{cases} .25 & 0 \leq y < 1 \\ .75 & 1 \leq y \leq 2 \end{cases}$ and you want to sample from it. You should get 25% of your samples between 0 and 1, and 75% of your samples between 1 and 2, correct? Does rejection sampling accomplish this?

Rejection Sampling

- 1 Define the same $q(y)$ and k as before, so that $kq(y) = 1$ from 0 to 2 (draw a picture to make sure it covers the target distribution)
- 2 Randomly generate a “candidate” sample, $\tilde{y}$, from $q(y)$
- 3 Flip a coin with probability of keeping $\tilde{y}$ that is equal to $\frac{p(\tilde{y})}{kq(\tilde{y})}$
 - For example, if $\tilde{y} = .3$, you would get $\frac{p(\tilde{y})}{kq(\tilde{y})} = \frac{.25}{1}$
 - For example, if $\tilde{y} = 1.6$, you would get $\frac{p(\tilde{y})}{kq(\tilde{y})} = \frac{.75}{1}$

Rejection Sampling

How does this “flipping of the coin” work?

- For a given $\tilde{y}$, randomly generate a number u from a uniform distribution between 0 and $kq(\tilde{y})$
- If $u < p(\tilde{y})$, accept $\tilde{y}$
- If $u \geq p(\tilde{y})$, reject $\tilde{y}$

Note: in order for you to accept, we have:

$$\begin{aligned} P(\tilde{y} \text{ sampled} \cap \tilde{y} \text{ accepted}) &= P(\tilde{y} \text{ sampled}) \times P(\tilde{y} \text{ accepted}) \\ &= q(\tilde{y}) \frac{p(\tilde{y})}{kq(\tilde{y})} = \frac{1}{k} p(\tilde{y}) \end{aligned}$$

Works for any $q(y)$

Reminder of the goal and steps:

- Goal is to generate samples from $p(y)$
- ① Pick a proposal distribution $q(y)$ and constant k such that $kq(y) > p(y)$
- ② Sample from the proposal distribution, call this sample $\tilde{y}$
- ③ Generate a uniform random value, $u \sim \text{Unif}(0, kq(\tilde{y}))$
 - We can do this in python very easily by: `k*q(y_prop)*rand()`
 - And in r by: `k*q(y_prop)*runif(1)`
- ④ If $u < p(\tilde{y})$, we accept the sample and go back to Step 1. If it is not, we throw it away and go back to Step 1.

Practice

See how it works in the **sampling_mcmc** files of both python and R.

Today

1 Sampling Review

2 Rejection

3 Ancestral

4 MCMC

5 MCMC: MH

6 MCMC: Gibbs

Ancestral Sampling

Ancestral sampling is the simplest method for sampling from **joint** distributions.

- Sometimes, as you will see on your homework, you might want/need to use rejection sampling as a **step** of ancestral sampling
- However, ancestral sampling can only be used in special situations where there is a **causal relationship** between the jointly varying features

Return to our example. You believe whether your crush texts you back (y) is **completely dependent** on the probability they are interested in you (θ):

$$p(y, \theta) = p(y|\theta)p(\theta)$$

- In this example, we know that $p(y|\theta)$ can only be known if we **first know** $p(\theta)$
- We also know that each of these distributions are common distributions that are easy to sample from (we chose them after all)

Ancestral Sampling

To perform ancestral sampling:

- 1 First sample from $p(\theta)$, which has the Beta distribution. Call those $i = 1, \dots, n$ samples $\tilde{\theta}_i$:

$$\tilde{\theta}_i \sim \text{Beta}(\alpha, \beta)$$

- This is easy to do in NumPy or R given the hyperparameters:

```
theta_samples = np.random.beta(alpha, beta, size=n)
theta_samples <- rbeta(n, alpha, beta)
```

- 2 Once you have samples of θ , plug them into $p(y|\theta)$ and sample from that distribution (the Bernoulli). Call those $i = 1, \dots, n$ samples $\tilde{y}_i$:

$$\tilde{y}_i | \tilde{\theta}_i \sim \text{Bern}(\tilde{\theta}_i)$$

- Easy to do in NumPy or R as well:

```
y_samples = np.random.binomial(n=1, theta_samples)
y_samples <- rbinom(n, size = 1, prob = theta_samples)
```

- 3 After these two steps, you have n samples from $p(y, \theta)$!

Practice

Follow along in **sampling_mcmc.ipynb** as we generate samples from the joint distribution of $p(y, \theta)$, and compare to the actual distribution (in this case, easy enough to find, as we know). Remember, we'll say:

$$y|\theta \sim \text{bern}(\theta)$$

$$\theta \sim \text{beta}(\alpha = 1, \beta = 19)$$

This corresponds to $E(\theta) = .05$, our prior belief that our crush is interested in us being 5%.

What do we do with these samples?

We can keep all the $\tilde{\theta}_i$ samples for $\tilde{y} = 0$ (or $\tilde{y} = 1$), and visualize the estimated posterior distribution with a histogram, and then **average** over those samples to get an estimate of the posterior mean!

- We'll talk about **why** we can do this next

Motivating with the Bernoulli-Beta

Return to our initial Bayesian example (this is the same slide as Slide 10).

$$y|\theta \sim \text{bern}(\theta)$$

$$\theta \sim \text{beta}(\alpha, \beta)$$

Where y was whether or not your crush texted you back ($y \in \{0, 1\}$) and θ was the probability they are interested in you. After some work, we showed that the posterior was:

$$p(\theta|y) = \frac{p(y, \theta)}{p(y)} = \frac{p(y|\theta)p(\theta)}{\int_{\theta} p(y, \theta)d\theta} =$$

$$\frac{\theta^y(1-\theta)^{1-y} \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1}(1-\theta)^{\beta-1}}{\int_{\theta} \theta^y(1-\theta)^{1-y} \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1}(1-\theta)^{\beta-1}d\theta} = \frac{\theta^{y+\alpha-1}(1-\theta)^{1-y+\beta-1}}{B(y+\alpha, 1-y+\beta)}$$

- **Important:** once you collect the data y , the evidence, since it has nothing to do with the unknown θ , **IS A CONSTANT**, in this case either $c_1 = p(y=0)$ or $c_2 = p(y=1)$
- If we can simulate data under $p(y, \theta)$, then we can estimate $p(y)$ simply by counting up the number of zeros and ones in our synthetic data!

How To Use The Samples

The previous slide reminded us that the integral for the evidence/marginal is $p(y)$ (a **constant**):

$$\int_{\theta} \theta^y (1 - \theta)^{1-y} \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1} (1 - \theta)^{\beta-1} d\theta = \mathbf{c} \text{ (some number)}$$

- Ultimately, this constant will be either $p(y = 1)$ or $p(y = 0)$
- Therefore, if we had the data, we could calculate these very simply:

$$p(y = 1) = \frac{\text{total 1s in the data}}{\text{total samples in the data}} \quad p(y = 0) = \frac{\text{total 0s in the data}}{\text{total samples in the data}}$$

- Therefore, when we want to know $p(\theta|y = 1)$ (for example), we can use our joint samples to look at the distribution of the $\tilde{\theta}$ samples **for only those $\tilde{y}$ samples which equal 1**

Practice

Take a look at the **sampling_mcmc** files to see us create histograms for the posterior distribution $\theta|y$, depending on if $y = 1$ or $y = 0$, then compare the estimated posterior means to the **true** posterior means we calculated back in Lecture 7. Recall that in this example:

- The data likelihood is $y|\theta \sim \text{Bern}(\theta)$
- The prior is $\theta \sim \text{Beta}(\alpha = 1, \beta = 19)$ which implies our prior mean is $E(\theta) = \frac{1}{1+19} = .05$
- The **true** posterior mean was $E(\theta|y) = \frac{y+\alpha}{1+\alpha+\beta}$, so:
 - If $y = 1$, $E(\theta|y = 1) = \frac{2}{21} = 0.095$
 - If $y = 0$, $E(\theta|y = 0) = \frac{1}{21} = 0.048$

Note What We've Done

We have just estimated a **conditional** distribution $p(\theta|y)$ by sampling from a **joint** distribution $p(y, \theta)$ and then adjusting depending on knowing the value of y . In other words, because we know:

$$p(\theta|y) \propto p(y, \theta)$$

This implies that we can simply sample from $p(y, \theta)$, then (for specific values of $y = y^*$), find:

$$p(\theta|y = y^*)$$

By simply scaling the joint samples by the (known) $p(y = y^*)$

Practice

In the **sampling_mcmc** files (either python or R) attempt to generate samples from the posterior $\mu|y = 48$ (of a single observation y) given:

$$y|\mu \sim N(\mu, \sigma = 1)$$

$$\mu \sim N(\eta = 50, \tau = 2)$$

- 1 Use rejection sampling to sample μ (I leave this to you)
- 2 You can simply use NumPy or base R to sample $y|\mu$ (I leave this to you)
- 3 Generate a heatmap of the samples from the joint distribution $p(y, \mu)$
- 4 Consider how we might use those samples to estimate $p(\mu|y = 48)$. What is different about this situation than the previous example?
- 5 Once you do estimate the posterior, compare it to the true posterior, since we know in this case it is:

$$\mu|y \sim N\left(\eta^* = \frac{\frac{48}{1^2} + \frac{50}{2^2}}{\frac{1}{1^2} + \frac{1}{2^2}}, \tau^* = \sqrt{\left(\frac{1}{1^2} + \frac{1}{2^2}\right)^{-1}}\right)$$

Today

1 Sampling Review

2 Rejection

3 Ancestral

4 MCMC

5 MCMC: MH

6 MCMC: Gibbs

MCMC: Markov Chain Monte Carlo

One of the most commonly used general methods for sampling from arbitrary posteriors, **Markov Chain Monte Carlo (MCMC)** proceeds generally by:

- 1 drawing values of θ from approximate distribution(s)
- 2 correcting those draws to better approximate the target posterior, $p(\theta|y)$

The sampling is done **sequentially**, with the distribution of the samples **dependent** on the last value drawn. This makes the sequence of draws something called a **Markov Chain**

Markov Chains

A **Markov Chain** is a sequence of random variables: $\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(t)}$ where, for any t the distribution function of $\theta^{(t)}$ depends only on the previous value of θ . Or, more succinctly:

$$p(\theta^{(t)} | \theta^{(t-1)}, \dots, \theta^{(1)}) = p(\theta^{(t)} | \theta^{(t-1)})$$

- In other words, for example, after initializing $\theta^{(0)}$:
 - you get $\theta^{(1)} | \theta^{(0)}$
 - then you get $\theta^{(2)} | \theta^{(1)}$ (you “forget” anything about $\theta^{(0)}$)

General MCMC

There are several (somewhat important) mathematical properties/requirements for Markov Chains which we won't get into. But, for our purposes, keep in mind:

- Markov chain simulations should be run for a **LONG** time; long enough that the distribution of the samples at time t are close enough to the true samples of the target distribution
- This is because they will **never** truly be samples from the target distribution, but that the distribution of $\theta^{(t)} | \theta^{(t-1)}$ will **converge** to the target distribution as t increases
- It is thus necessary to verify convergence, which can be done in multiple ways. The easiest (which we'll do) is to inspect time series plots of the iterative samples.
 - The BDA textbook has a better way to do this (page 284-285), though it is a bit tedious.

General MCMC

- Because it often takes quite some time to converge, we often **discard** the first so many samples (also called “**burn-in**”)
 - The choice of burn-in size (how many of the first samples to discard) varies on how quickly the algorithm is converging, but there is generally an 18% rule of thumb
- While we assume the Markov Chain is supposed to be **memoryless** (remember, you forget about the samples before time $t - 1$) we can **thin** the chain to remove potentially correlated samples (more on this later)
- The two most basic MCMC methods are:
 - Metropolis Hastings (Metropolis)
 - Gibbs Sampling

Today

1 Sampling Review

2 Rejection

3 Ancestral

4 MCMC

5 MCMC: MH

6 MCMC: Gibbs

Metropolis Hastings Sampling vs. Ancestral Sampling

Why do we need MCMC methods if we already have Ancestral Sampling?

- Ancestral sampling **only works** if our joint distribution has a very specific structure of dependency, for example:

$$p(x, y, z) = p(x|y)p(y|z)p(z)$$

- Where we can cleanly separate out the joint distribution into simple conditional distributions
- But what happens if we cannot do this? For example, how do we generate samples from something like:

$$p(y) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(y - \mu)^T \Sigma^{-1}(y - \mu)\right) \quad \text{where } y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \end{bmatrix}$$

Metropolis Sampling

Metropolis sampling still works for distributions that cannot be separated into ancestral patterns

- That is, sampling directly from $p(x, y, z)$ without knowing how to split them into conditionals
- Or, if we happen to have other non-ancestral forms like $p(x_1, x_2)p(x_3, x_4)p(x_5)$

Sampling with Metropolis Hastings

Metropolis Hastings is one of the most impactful sampling algorithms in history, and simple to implement (Dr. Gerber has even done it),

Metropolis Hastings Set Up

- Identify the target distribution $p(y)$ where $y \in \mathbb{R}^d$
- Choose a **proposal distribution**, $q(y)$ of the same dimension ($y \in \mathbb{R}^d$)
 - The Isotropic Gaussian is almost always chosen as $q(y)$
- Choose an initial sample, $\tilde{y}_0$
- You're ready to start...

Sampling with Metropolis Hastings

Assume $\tilde{y}_0$ was randomly assigned and comes from $q(y)$ (the proposal distribution; which we will eventually use the Isotropic Gaussian):

Step 1. Sample from $q(y|\mu = \tilde{y}_0)$ to obtain a **candidate** sample $\tilde{y}'$

Step 2. Calculate the probability of accepting $\tilde{y}'$ with

$$\alpha = \frac{p(\tilde{y}')}{p(\tilde{y}_0)} \frac{q(\tilde{y}_0|\mu = \tilde{y}')}{q(\tilde{y}'|\mu = \tilde{y}_0)}$$

Step 3. Generate an acceptance sample from a **Uniform distribution**

$$u \sim \text{Unif}(0, 1)$$

Step 4. If $u < \alpha$: accept y_tilde'
else: reject

- **Note** that if we accept $\tilde{y}'$, it officially becomes our first sample $\tilde{y}_1$. If we reject, we **re-record the old sample as $\tilde{y}_1$** ! This is different from rejection sampling!

Step 5/1. To sample the next candidate, $\tilde{y}' \sim q(y|\mu = \tilde{y}_1)$, then repeat

- Note that we are now using the previous sample as the mean of our proposal distribution

Sampling with Metropolis Hastings

Once the Markov chain has converged (we'll assess via plots):

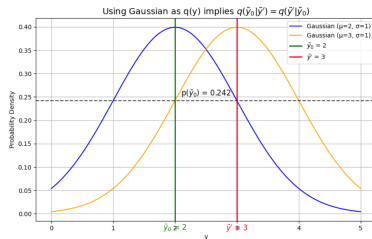
- Remove the burn-in samples (rule of thumb is $\approx 18\%$, but can use the plots to guide)
- **Advanced:** check the autocorrelation plots of the remaining (non burn-in) samples; if there is strong autocorrelation, consider **thinning** (discarding every other, or every third, e.g.) the chain
 - You'll want to have **lots** of samples before you do this

When this is done, your samples can be used to approximate the target distribution and any integrals of interest involving it.

Why Use the Isotropic Gaussian? (Enter Metropolis)

The Isotropic Gaussian is the post popular proposal distribution, $q(y)$, because it has an algorithmic benefit. Consider the below plot (also created in `sampling_mcmc.ipynb`):

- Set $\tilde{y}_0 = 2$ and $\tilde{y}' = 3$
- The likelihoods
 $q(\tilde{y}_0 | \mu = \tilde{y}') = q(\tilde{y}' | \mu = \tilde{y}_0)$
when they are normal with the same variance
- Therefore, an Isotropic Gaussian simplifies the α acceptance ratio to: $\alpha = \frac{p(\tilde{y}')}{p(\tilde{y}_0)}$
- When this happens, we call this a **Metropolis** algorithm (not Metropolis-Hastings)



General Rules for Selecting σ

If we are to propose from an Isotropic Gaussian, we will need to choose σ efficiently:

$$q(y) = \frac{1}{(2\pi\sigma^2)^{d/2}} \exp\left(-\frac{\|y - \mu\|^2}{2\sigma^2}\right)$$

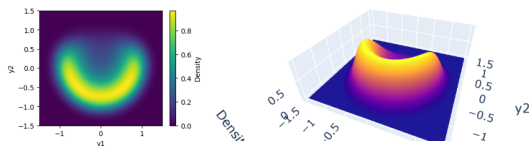
- 1 Aim for an acceptance ratio α of 20-50%:
 - Optimally: $\approx 23.4\%$ for high-dimensional problems
 - If $\alpha > .5$, the acceptance is too high: σ is **too small**, causing slow exploration (you will also see high autocorrelation in the ACF plot)
 - If $\alpha > .5$, **increase** σ
 - If $\alpha < .2$, the acceptance is too low: σ is **too large**, leading to frequent rejections
 - If $\alpha < .2$, **decrease** σ
- 2 You can initialize/have some intuition about σ based on the **target distribution**
 - If the target distribution, $p(y)$, is **narrow**, choose a **smaller** σ
 - If the target distribution, $p(y)$, is **broad**, choose a **larger** σ

Example

The Banana distribution is a special type of Bivariate Gaussian. Suppose we have a Banana distribution with probability function:

$$p(y_1, y_2) = \exp \left(-\frac{(y_1^2 + y_2^2 - .5)^2}{0.5} - \frac{(y_2 - y_1^2 + 1)^2}{1.5} \right)$$

The heatmap and 3D plots look like this:



- 1 Using the starter code (I've done most of it, but you will need to update the Metropolis function) in either python or R, generate 10000 samples from this distribution using the Metropolis algorithm
 - Discard 18% of the initial samples as burn-in
 - Assess the ACF plot to see if the chain is mixing well
- 2 Plot the histogram or the heatmap of the final samples and compare to the true distribution

Notes on the Metropolis

- If you run the code a second (or third) time, you'll note that the histogram changes shape each time
 - This implies we don't have enough samples
- We also saw that there was strong autocorrelation in the samples
 - This implies we don't have independent samples, and may want to thin (meaning we want more initial samples)
- Metropolis is simple, and can be powerful, but we need a lot of samples to accurately represent the distribution

Practice: Metropolis with Bayes

Suppose we have bivariate normal data with unknown mean vector and known covariance matrix:

$$y|\mu \sim MVN_2 \left(\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \Sigma = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \right)$$

We already know that the multivariate normal is a conjugate prior for the unknown means:

$$\mu \sim MVN_2(\mu_0, \Sigma_0)$$

And we already know the posterior (so we can see if our samples are approximating it well):

$$\mu|y \sim MVN_2 \left(\mu^* = (\Sigma_0^{-1} + \Sigma^{-1})^{-1}(\Sigma_0^{-1}\mu_0 + \Sigma^{-1}y), \Sigma^* = (\Sigma_0^{-1} + \Sigma^{-1})^{-1} \right)$$

Practice: Metropolis with Bayes

Try it on your own

Use the starter code to practice this in the **sampling_mcmc** files.

Generate samples from the joint distribution:

$$p(y, \mu)$$

And then compare the true posterior to the estimated posterior distribution for

$\mu|y = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$ given prior values:

- $\mu_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- $\Sigma_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

Today

1 Sampling Review

2 Rejection

3 Ancestral

4 MCMC

5 MCMC: MH

6 MCMC: Gibbs

Gibbs Sampling

Gibbs Sampling can be seen as a special case of Metropolis-Hastings where:

- It generally requires fewer samples
- It is harder to implement, **especially** if we don't have nicely separable distributions, e.g:

$p(x_1, x_2, x_3) \Rightarrow$ easy for Metropolis, hard for Gibbs

$p(x_1)p(x_2)p(x_3) \Rightarrow$ easy enough for both

- In cases where it is easy to implement Gibbs, it is preferred since it is more efficient

Sampling with Gibbs

Gibbs Set Up

- Identify the **joint** target distribution (e.g. $p(y, \theta)$, but for illustration purposes, let's use a more general $p(y_1, y_2, y_3)$ as an example)
- Choose initial values for each unknown, e.g. $\tilde{y}_{10} = a_0$, $\tilde{y}_{20} = b_0$, and $\tilde{y}_{30} = c_0$
- You're ready to start...

Sampling with Gibbs

Assume the vector $[a_0 \quad b_0 \quad c_0]$ represent a random sample from the joint distribution $p(y_1, y_2, y_3)$:

- Step 1.** Sample the next value of y_1 from $a_1 \sim p(y_1 | y_2 = b_0, y_3 = c_0)$
 - Step 2.** Sample the next value of y_2 from $b_1 \sim p(y_2 | y_1 = a_1, y_3 = c_0)$
 - Step 3.** Sample the next value of y_3 from $c_1 \sim p(y_3 | y_1 = a_1, y_2 = b_1)$
 - Step 4.** Save the vector $[a_1 \quad b_1 \quad c_1]$ as the next sample from the joint distribution and return to Step 1.
- End.** When you have finished drawing all your samples, once again discard $\approx 18\%$ of the samples as burn-in.

Practice

A self-driving car relies on multiple sensors to estimate the position (y_1, y_2) of a pedestrian relative to the car. Due to sensor noise, the estimated position follows a bivariate Gaussian:

$$y_1, y_2 \sim MVN_2 \left(\mu = \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \Sigma = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \right)$$

- 1 Use Gibbs sampling to generate samples from this distribution
 - You would need to iterate between $p(y_1|y_2)$ and $p(y_2|y_1)$
- 2 Once you generate the samples, estimate μ and Σ
- 3 They should be very close to the true values of the distribution

Gibbs in a Bayesian Context

These MCMC samplers are not restricted to use in Bayesian models, but they are commonly used as such. In a Bayesian context, remember our goal is to sample from the posterior of some number of unknown parameters, say $\theta = [\theta_1 \ \theta_2 \ \cdots \ \theta_d]$ where we know the likelihood and the prior can be used to find the joint distribution:

$$p(y, \theta_1, \theta_2, \cdots, \theta_d) = p(y|\theta)p(\theta)$$

- Goal: Estimate $p(\theta_1, \theta_2, \cdots, \theta_d|y)$

The Gibbs Sampler will iteratively sample from the conditional distributions of each θ_j given all the other θ_{j-} (or, the set of θ without θ_j) and the data:

$$\tilde{\theta}_j \sim p(\theta_j|\tilde{\theta}_{j-}, y)$$

Note: this often means we **do** need to be able to sample from the **conditional posterior** of a single parameter.